SIM-003

MQTT Telemetry Simulator

Battery telemetry simulation for safe cybersecurity testing

Internship Project | Vishal Kumar

B.Tech Electronics & Communication Engineering

Agenda

1. Background and problem
2. Objectives and scope
3. Technology and architecture
4. Data model and 7 scenarios
5. API, dashboard and command line
6. Testing, security and limits
7. Conclusion and future work

Introduction and Need

- Battery telemetry: voltage, current, temperature,
- state of charge (SOC), state of health (SOH)
- Real BMS setup needs hardware and sensors
- Not always available in early development
- A software simulator gives repeatable test data
- Users: developers and testers of telemetry trust

Problem and Objectives

Problem

Testers need normal AND abnormal battery events: delayed, duplicate, fake, malformed, replayed.

Manual data or hardware is slow and hard to repeat.

Objectives

Generate simulated telemetry

Support all 7 scenarios

Flag abnormal events

FastAPI + Streamlit interfaces

CSV export

Automated tests and docs

Project Scope

In Scope

Battery telemetry generation

Scenario-based events

Event metadata (ID, topic, QoS)

FastAPI and Streamlit

pytest tests and coverage

QA and security documents

Out of Scope

Real MQTT broker

Physical BMS or sensors

Real network traffic

Real attacks

Production-grade security

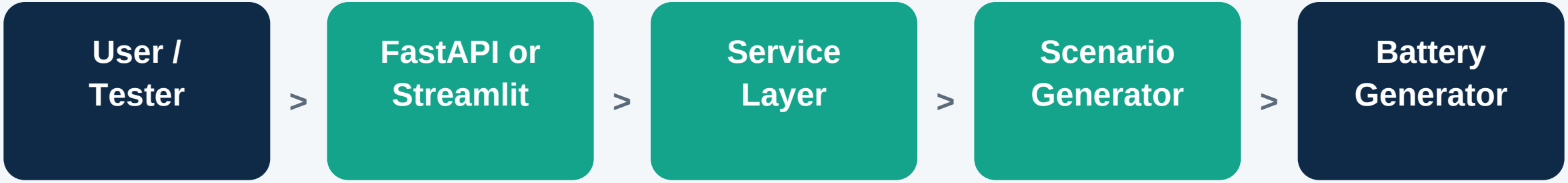
Proprietary BMS protocols

Technology Stack

Python 3.10+	FastAPI	Streamlit	pytest	Git / GitHub
Main language	REST API	Dashboard	Automated tests	Source control
Pydantic v2	Uvicorn	Pandas	pytest-cov	Branch + PR
Data validation	API server	Tables and CSV	Coverage	Project history

Modular code: logic is separate from API and UI

High-Level Architecture



Output: JSON response | Streamlit table | CSV file

Topic pattern: battery/telemetry/{battery_id}

Every event carries simulated = true

Module Design

models.py

BatteryData model

generator.py

Creates battery values

scenarios.py

7 scenarios engine

service.py

Shared validation

api.py

FastAPI endpoints

app.py

Streamlit dashboard

main.py

Command-line tool

test/

Automated tests

Telemetry Data Model

Battery Data

battery_id

voltage, current

temperature

soc, soh

timestamp

Added by Scenario

event_id (EVT-001)

sequence_number

topic and qos

simulated = true

anomaly flag, sent_at

Structured, JSON-compatible, validated with Pydantic

The 7 Scenarios

Normal

Baseline case

Delayed

Sent 1-300 sec late

Duplicate

Same sequence no.

Out of Range

65 V, 75 C, 600 A

Missing Timestamp

Timestamp removed

Spoofed Identity

Fake SIM-SPOOF ID

Replay Attack

Old timestamp

Each abnormal case gets an anomaly flag

Replay Attack Scenario

Telemetry time	Sent time	Result
Moved back 300 seconds	sent_at = now (current time)	anomaly = REPLAY_DETECTED

- Old data sent again looks like a replay
- Visible in the generated JSON
- Tested in Swagger and Streamlit demo

FastAPI Endpoints

GET /	API is running
GET /health	Healthy status
GET /scenarios	List 7 scenarios
GET /sample	Sample normal event
POST /generate-events	Generate telemetry

Example request for the replay scenario

(shown on the right)

```
POST /generate-events
{
  "battery_id": "BAT001",
  "scenario": "replay_attack",
  "count": 2,
  "delay_seconds": 10
}
```

Dashboard and Command Line

Streamlit Dashboard

Sidebar: Battery ID, Scenario,

Number of Events, Delay

Click Generate Events

Event table (Pandas)

Metrics: events, anomalies

CSV download

Command Line (main.py)

Give an event count

Generates telemetry

Saves sample JSON to

sample_data/battery_data.json

Testing and QA Results

19
Tests Passed

0
Tests Failed

71%
Code Coverage

- Covers generator, service and API layers
- Checks replay, validation errors, health, samples
- Only 1 minor deprecation warning (no impact)

Security and Limitations

Security Design

Safe local simulation

simulated = true on all events

Attacks shown as data only

No real MQTT traffic

No real devices touched

Limitations

No real MQTT transport

No physical BMS or sensors

Not production-grade security

External cross-testing not done

Best seen as a prototype

Conclusion and Future Work

Achieved

7 scenarios working

Service, API, dashboard

19 tests, 71% coverage

Docs, demo, PR merged

Future Work

Real MQTT broker

Automatic replay detection

Richer dashboard charts

Performance testing

Learned

Python, Pydantic

FastAPI, Streamlit

pytest, Git/GitHub

Safe security simulation

Thank you! Any questions?